PRACTICAL-08

AIM: Implementation of Graph and Searching (DFS and BFS).

ALGORITHM:

Input : Graph with n vertices, number of edges edges (u,v) and starting vertex start.

output : Adjacency matrix, DFS traversal and BFS traversal.

Step 1: Start
Step 2: Read n, the number of vertices
Step 3: Set adj[i][i] =0 for i=0 to n-1 and j=0 to n-1.
Step 4: Read the number of edges.
Step 5: Repeat all the edges (Read vertices u and v.
Step 6: set adj[u][v] =1 and adj[v][u] =1.
Step 7: Display the adjacency matrix.
Step 8: Read the starting vertex start.
Step 9: Set visited [i]=0 for i=0 to n-1
Step 10: Set visited [start]=1 and print start.
Step 11: Explore the vertex by checking i=0 to n-1
Step 12: If adj [start][i]=1 and visited [i]=0, call DFS for vertex i.
Step 13: Repeat Step 11 and step 12 for all adjacent vertices.
Step 14: Repeat visited [i]=0 to i=0 to n-1

Step 15: set front=0 and rear=0
Step 16: Insert start into the queue and set visited [start]=1.
Step 17: Repeat while front<rear.
Step 18: Remove the vertex from the queue and store it in vertex.
Step 19: print vertex.
Step 20: Explore the vertex by checking i=0 to n-1
Step 21: If adj [vertex][i]=1 and visited[i] =0, set visited [i]=1 and insert i into the queue.
Step 22: Repeat step 18 to step 21 until the queue becmes empty.
Step 23: print DFS and BFS traversal
Step 24: Stop.

PROGRAM:

```
#include <iostream>
using namespace std;
class Graph
{ public: int adj[10][10];
 int visited[10];
 int n; void getInput()
 {
    int edges;
    cout << "Enter number of vertices: ";
    cin >> n; for(int i = 0; i < n; i++)
       {
          for(int j = 0; j < n; j++)
             {
                adj[i][j] = 0;
             }
       } cout << "Enter number of edges: ";
    cin >> edges;
    cout << "Enter edges:\n";
    for(int i = 0; i < edges; i++)
    {
       int u, v;
       cin >> u >> v;
       adj[u][v] = 1;
       adj[v][u] = 1;
    } }
 void displayGraph()
```

```
{
    cout << "\nAdjacency Matrix:\n";
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cout << adj[i][j] << " ";
        }
        cout << endl;
    } }
void DFS(int start)
{
    visited[start] = 1;
    cout << start << " ";
    for(int i = 0; i < n; i++){
        if(adj[start][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}
void BFS(int start)
{
    int queue[10];
    int front = 0;
    int rear = 0;
    for(int i = 0; i < n; i++)
        {
            visited[i] = 0;
```

```
        }
    visited[start] = 1;
    queue[rear] = start;
    rear++;

    while(front < rear) {
        int vertex = queue[front];
        front++;
        cout << vertex << " ";
        for(int i = 0; i < n; i++)
        {
            if(adj[vertex][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                queue[rear] = i;
                rear++;
            }
        }
    }
  }
};
int main()
{
    Graph g;
    int start;
    g.getInput();
    g.displayGraph();
    cout << "\nEnter starting vertex: ";
    cin >> start;
```

```
    for(int i = 0; i < g.n; i++)
        {
         g.visited[i] = 0;
        }
    cout << "\nDFS Traversal: ";
    g.DFS(start);
    cout << "\nBFS Traversal: ";
    g.BFS(start);
    cout << endl;
    return 0;
}
```

OUTPUT:

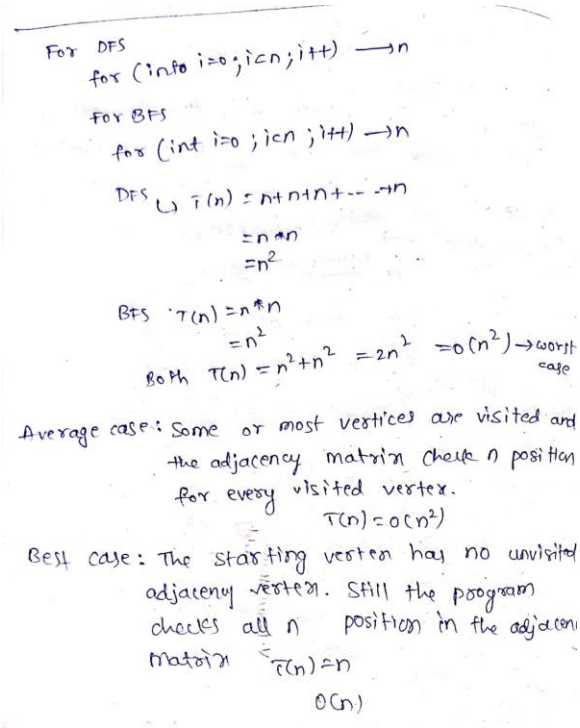
```
Enter number of vertices: 5
Enter number of edges: 4
Enter edges:
1
2
0
2
4
3
6
7

Adjacency Matrix:
0 0 1 0 0
0 0 1 0 0
1 1 0 0 0
0 0 0 0 1
0 0 0 1 0
```

```
Enter starting vertex: 0

DFS Traversal: 0 2 1
BFS Traversal: 0 2 1
```

TIME COMPLEXITY:



CONCLUSION: Thus, the Graph was successfully implemented using an adjacency matrix, and the DFS and BFS searching techniques were performed successfully. DFS explores the graph deeply using recursion, while BFS explores the graph level by level using a queue. The time complexity of the traversal is O(n) in the best case and $O(n^2)$ in the average and worst cases for this implementation.